

UVM-Based Verification of AXI4-Lite to APB Bus Bridge Protocol

Verification Project Report

Submitted by

Muhammad Hammad
Zain-ul-Aabideen
Mirza Muhammad Umar

Supervisor

Engr. Abdur-Rafy

1. Introduction

This report covers the UVM-based verification of an AXI4-Lite to APB bridge. AXI4-Lite is the lightweight, register-access subset of AXI4, and APB is the simple peripheral bus used inside most SoCs for control and status registers. Because the two protocols follow completely different timing models, a bridge is needed to translate between them, and this project verifies that the bridge does that translation correctly. The verification itself is built as a UVM environment: it drives AXI4-Lite transactions into the bridge, observes the resulting APB activity, and checks that every read, write, reset, delayed response, and error condition behaves the way the specification says it should.

2. Protocol Basics

2.1 AXI4-Lite Protocol

AXI4-Lite is a simplified version of AXI4, built for lightweight access to control and status registers rather than high-throughput data movement. The key difference from full AXI4 is that AXI4-Lite supports only single-beat transactions - one data transfer per read or per write, with no bursts. It still uses the same five independent channels as AXI4, each governed by its own VALID/READY handshake, but it drops the burst, ID, and cache/lock signals that AXI4 needs for multi-beat and out-of-order transfers.

- 1) Read address channel
- 2) Read data channel
- 3) Write address channel
- 4) Write data channel
- 5) Write response channel

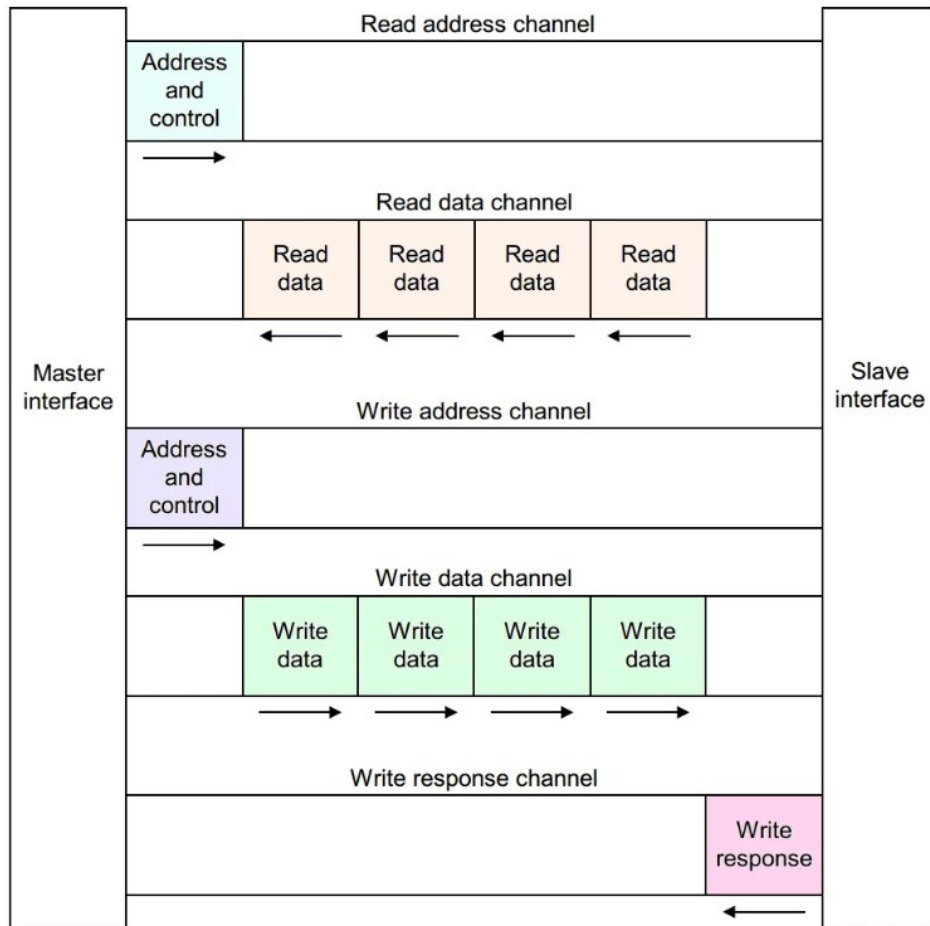


Figure 1: The five channels shared by AXI4-Lite.

2.2 AXI4-Lite Channel Signals

Because AXI4-Lite drops burst and ID handling, its signal set is much smaller than full AXI4. Each channel carries only an address/data payload, an optional protection field, and the VALID/READY handshake pair:

Channel	Signal	Direction	Description
Write Address	AWADDR	M -> S	Write address
Write Address	AWPROT	M -> S	Protection/access type (optional)
Write Address	AWVALID	M -> S	Address is valid
Write Address	AWREADY	S -> M	Slave ready to accept the address
Write Data	WDATA	M -> S	Write data
Write Data	WSTRB	M -> S	Byte-lane write strobes
Write Data	WVALID	M -> S	Data is valid
Write Data	WREADY	S -> M	Slave ready to accept the data

Channel	Signal	Direction	Description
Write Response	BRESP	S -> M	Write response (OKAY / SLVERR)
Write Response	BVALID	S -> M	Response is valid
Write Response	BREADY	M -> S	Master ready to accept the response
Read Address	ARADDR	M -> S	Read address
Read Address	ARPROT	M -> S	Protection/access type (optional)
Read Address	ARVALID	M -> S	Address is valid
Read Address	ARREADY	S -> M	Slave ready to accept the address
Read Data	RDATA	S -> M	Read data
Read Data	RRESP	S -> M	Read response (OKAY / SLVERR)
Read Data	RVALID	S -> M	Data is valid
Read Data	RREADY	M -> S	Master ready to accept the data

Table 1: AXI4-Lite signals by channel (M = master, S = slave).

2.3 APB Protocol

APB (Advanced Peripheral Bus) is the low-power, low-complexity member of the AMBA family, meant for simple peripherals - UARTs, GPIO, timers, and other control/status registers - that don't need AXI's throughput. Every APB transfer runs in two phases:

- **Setup phase** - the address (PADDR), control signals (PWRITE, PSEL) and, for a write, the data (PDATA) are all placed on the bus.
- **Enable phase** - PENABLE is asserted one cycle later, which is when the actual data transfer or read response takes place.

A slave can hold PREADY low to insert wait states, or set PSLVERR to flag an error on the current transfer.

2.4 Need for Bridging

AXI4-Lite and APB belong to the same AMBA family but do not operate the same way. AXI4-Lite uses independent channels with a VALID/READY handshake, so address and data can move in the same cycle, while APB is strictly sequential - setup, then enable, one transfer at a time. Connecting an AXI4-Lite master directly to an APB slave would produce mismatched timing and undefined control sequences, so a bridge sits between them: it accepts a read or write from the AXI4-Lite side, holds the address and data, and then drives the equivalent APB sequence on the other side, preserving both protocols' timing rules.

3. Bridge Design (Design Under Test)

3.1 Design Overview

The bridge is organized into three functional blocks:

- **AXI4-Lite Interface Unit** - accepts read and write requests from the AXI4-Lite master and manages its VALID/READY handshakes.
- **APB Interface Unit** - generates the PADDR / PWRITE / PSEL / PENABLE / PWDATA sequence for the APB slave.
- **Control Unit (FSM)** - sits between the two interfaces and sequences the handshakes so an AXI4-Lite transaction only completes once the matching APB transfer has finished.

Because AXI4-Lite is restricted to single-beat transactions, the bridge only ever has to convert one read or one write at a time - there is no burst splitting and no need for a FIFO to hold multiple outstanding beats.

3.2 Design Flow

Write transaction:

- The AXI4-Lite master asserts AWVALID and WVALID together with the address and data.
- The bridge captures both and drives PADDR, PWDATA, PWRITE and PSEL, followed by PENABLE, on the APB side.
- Once the APB slave returns PREADY, the bridge asserts BVALID on the write response channel.

Read transaction:

- The AXI4-Lite master asserts ARVALID together with ARADDR.
- The bridge activates PSEL and PENABLE on the APB side to read the target register.
- Once PRDATA is returned, the bridge forwards it to the master on RDATA together with RVALID.

3.3 Finite State Machine (FSM) Design

The bridge's control logic is a single finite state machine running off one system clock. It steps through setup, transfer, wait, and response states on both the read and write paths, with every transition gated by the relevant handshake signal - VALID, READY, or PREADY - and a synchronous reset that returns the FSM to a known idle state.

4. UVM Verification Methodology

UVM (Universal Verification Methodology) is a standardized, SystemVerilog class library for building reusable, constrained-random testbenches. Rather than writing a fresh testbench for every test, a UVM environment is built once from a set of standard components and reused across many tests:

- **Sequence / Sequencer** - generates the stream of AXI4-Lite transactions, directed or randomized, that drive each test.
- **Driver** - converts each transaction into pin-level activity on the AXI4-Lite interface.
- **Monitor** - passively observes the AXI4-Lite and APB interfaces and packages the activity into transactions for checking.
- **Scoreboard** - compares what the bridge actually did on APB against what a reference model expected, and flags any mismatch.
- **Agent / Environment** - bundle the driver, sequencer and monitor together, and combine the AXI4-Lite-side and APB-side agents into the complete testbench.

This reusable structure is what makes the verification methodology in the next section practical - the same environment covers reset, directed read/write tests, wait-state and error corner cases, and random stress testing, without rewriting the testbench for each one.

5. Verification Environment & Methodology

5.1 Verification Environment

The UVM testbench was simulated in ModelSim/QuestaSim, built around two agents plus a scoreboard and reference model, as shown in Figure 2:

- **AXI4-Lite Agent** - its sequencer generates read/write transactions and its driver applies them to the bridge's AXI4-Lite interface.
- **Bridge Under Test (BUT)** - the RTL implementation of the AXI4-Lite-to-APB bridge.
- **APB Agent** - its driver models the APB slave, responding with PRDATA/PREADY (including configurable wait states and errors), while its monitor observes the resulting bus activity.
- **Scoreboard / Reference Model** - compares the actual APB-side activity against what the reference model expected for each AXI4-Lite transaction, and flags any mismatch automatically.

Because checking happens in the scoreboard rather than by eye, the testbench is self-checking - a test passes or fails on its own, without anyone needing to read the waveform by hand.

AXI4-Lite MASTER → Bridge_Complete (DUT) → APB SLAVE

Top-Level Architecture & Signal Flow

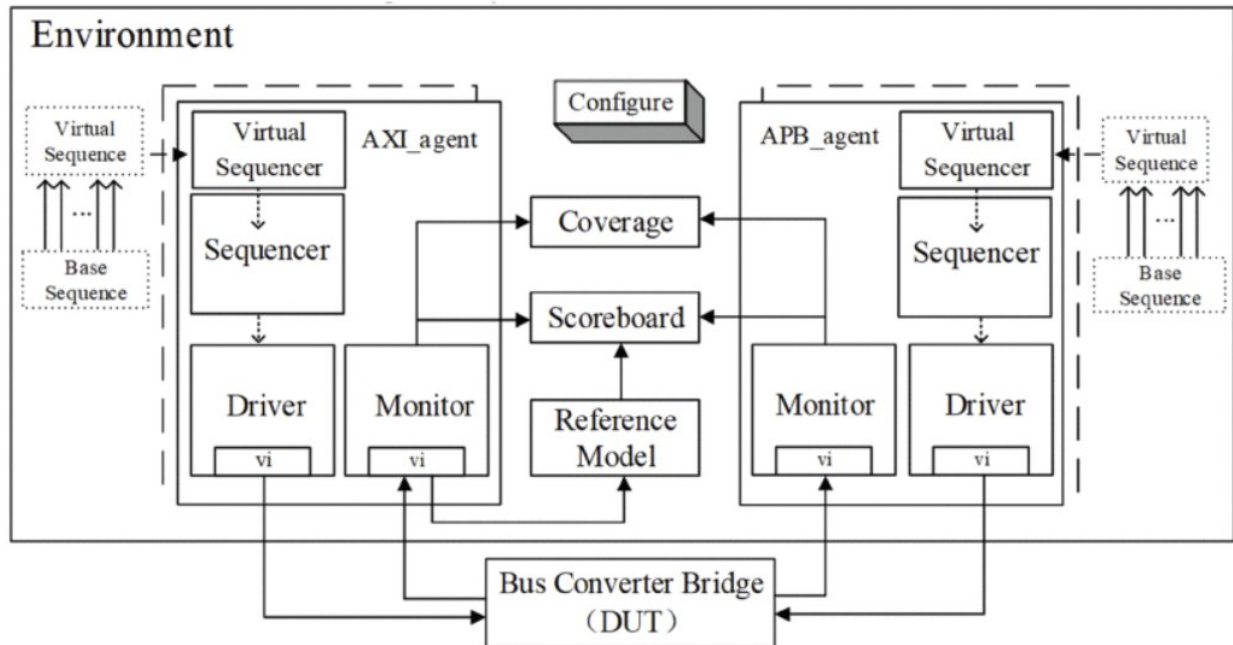
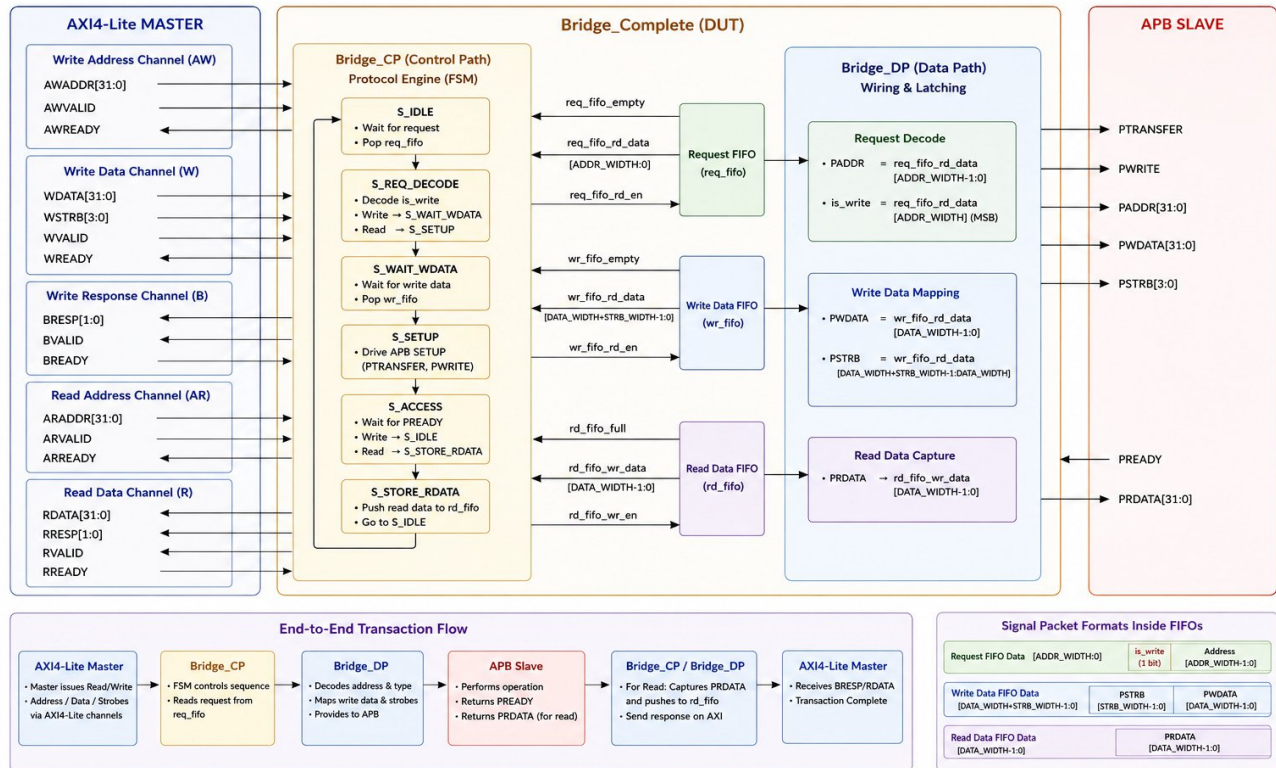


Figure 2: UVM verification environment - AXI4-Lite and APB agents around the bridge (DUT).

5.2 Test Scenarios

- **Reset phase** - reset is asserted first, and the FSM and all internal registers are checked to confirm they return to a known idle state.
- **Write transaction testing** - AWVALID/WVALID are driven with address and data; the bridge is checked to produce the correct PADDR/PWRITE/PWDATA/PSEL/PENABLE sequence, and to return BVALID only after PREADY.
- **Read transaction testing** - ARVALID/ARADDR are driven; the bridge is checked to produce the correct PSEL/PENABLE sequence, and to return RDATA/RVALID only after PRDATA is valid.
- **Wait-state and error handling** - the APB agent holds PREADY low to confirm the bridge waits correctly, and asserts PSLVERR to confirm the error is reported back to the AXI4-Lite master.

5.3 Result Checking

Every test runs through the scoreboard automatically, and functional coverage is collected alongside it to confirm that both the read and write paths, wait-state handling, and error responses were all actually exercised. Waveforms captured in Cadence Xcelium were used only for debugging failures, not for pass/fail decisions. Across all scenarios, the handshake signals on both sides stayed correctly synchronized, and no timing violations or missed transfers were observed.

6. Verification Test Plan

The table below summarizes the tests used to exercise the bridge, from reset and basic single-beat transfers through to error handling and a random stress run.

Test Name	AXI4-Lite Sequence	APB Slave Behaviour	What Is Verified
Reset Test	Reset assertion	Normal	FSM and internal registers return to a known idle state
Write Test	Single write	Normal	AXI4-Lite write correctly translated to an APB write
Read Test	Single read	Returns data	APB read data correctly returned on the RDATA channel
Random Test	Random single-beat reads/writes	Normal	General functional correctness under mixed traffic
Wait-State Test	Read/write	Delayed PREADY	Bridge correctly waits on the APB handshake
Error Test	Read/write	PSLVERR = 1	APB error is correctly propagated back to AXI4-Lite
Stress Test	Many random transactions	Random delays/errors	Overall robustness of the bridge

7. Conclusion

Verifying an AXI4-Lite to APB bridge comes down to making sure two very different timing models - AXI4-Lite's parallel, channel-based handshake and APB's sequential setup/enable cycle - are translated correctly in both directions. Building the checks in UVM meant the same environment could be reused across reset, normal read/write, wait-state, and error scenarios instead of writing a new testbench for each one. Across every scenario tested, the bridge produced the correct APB sequence for each AXI4-Lite transaction and returned the correct response, with no functional or timing issues observed, giving reasonable confidence that the design is ready for integration into a larger SoC.